

P2457R0

2021 December Library Evolution Poll Outcomes

Published Proposal, 2022-01-14

Author:

[Bryce Adelstein Leibach \(he/him/his\)](#) — [Library Evolution Chair](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

3.1 Poll 1: [P0009R14] `mdspan`

3.2 Poll 2: [P2093R11] Formatted Output

3.3 Poll 3: [P1467R7] Extended Floating-Point Types

3.4 Poll 4: [P2278R2] `cbegin` Should Always Return A Constant Iterator

3.5 Poll 5: [P2408R4] Ranges Iterators As Inputs To Non-Ranges Algorithms

3.6 Poll 6: [P2374R3] `views::cartesian_product`

3.7 Poll 7: [P2441R1] `views::join_with`

- 3.8 Poll 8: [P2446R1] `views::all_move`
- 3.9 Poll 9: [P2467R0] Support Exclusive Mode For `fstreams`
- 3.10 Poll 10: [P2438R1] `string::substr()` &&
- 3.11 Poll 11: [P2396R0] Concurrency TS v2 Fixes
- 3.12 Poll 12: [P2198R3] Freestanding Feature-Test Macros And Implementation-Defined Extensions C++23

References

Informative References

§ 1. Introduction

In December 2021, the C++ Library Evolution group conducted a series of electronic decision polls [P2456R0]. This paper provides the results of those polls and summarizes the results.

In total, 36 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send [P0009R14] (mdspan) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	19	9	1	0	0	Strong consensus in favor.
Poll 2: Send [P2093R11] (Formatted Output) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	18	9	2	0	1	Consensus in favor.

Poll 3: Send [P1467R7] (Extended Floating-Point Types) to Evolution Working Group and Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	10	15	0	2	0	Consensus in favor.
Poll 4: Send [P2278R2] (cbegin Should Always Return A Constant Iterator) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	10	12	2	2	0	Consensus in favor.
Poll 5: Send [P2408R4] (Ranges Iterators As Inputs To Non-Ranges Algorithms) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	9	11	2	0	0	Strong consensus in favor.
Poll 6: Send [P2374R3] (views::cartesian_product) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	14	9	0	2	0	Consensus in favor.
Poll 7: Send [P2441R1] (views::join_with) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	12	7	0	1	0	Consensus in favor.
Poll 8: Send [P2446R1] (views::all_move) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	9	9	1	3	0	Consensus in favor.
Poll 9: Send [P2467R0] (Support Exclusive Mode For fstreams) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	13	10	1	0	0	Strong consensus in favor.
Poll 10: Send [P2438R1] (string::substr() &&) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	7	18	1	0	0	Strong consensus in favor.
Poll 11: Send [P2396R0] (Concurrency TS v2 Fixes) to Library Working Group for the	10	12	0	0	0	Unanimous consensus in

Concurrency TS v2, classified as an addition ([P0592R4] bucket 3 item).						favor.
Poll 12: Adopt the freestanding feature test macro policy from [P2198R3] (Freestanding Feature-Test Macros And Implementation-Defined Extensions C++23).	8	10	1	2	2	Consensus in favor.

§ 3. Selected Poll Comments

§ 3.1. Poll 1: [P0009R14]

Having a multidimensional mapping by itself is very useful, but the additional customization point, allow the usage beyond that. For example, the accessor can be used to make shared- view over array.

— Strongly Favor

While a very basic implementation of this general idea is trivial, having it defined in the standard library supports its use in important applications like the BLAS proposal and also fosters interoperability among numeric components. The additional features of static extents and accessors are less trivial and significantly improve performance in important use cases.

— Strongly Favor

Time for this to progress. It's been in work since well before 2017.

— Weakly Favor

Greatly improved by the addition of multi-parameter operator[].

— Weakly Favor

I've been following this since Urbana. Much of the early parts of this did not inspire me with confidence, but I suppose the time has finally come.

— Neutral

§ 3.2. Poll 2: [P2093R11] Formatted Output

`std::print` includes all the goodness of `std::format`, and lets us output directly, without creating a temporary string. It's what we should have had a decade or more ago. Please, let's standardize it and move on with our lives.

— Strongly Favor

I still have mild concerns about prescribing a behavior for a precondition violation, but I have no doubt this will be one of the most impact-full feature of C++23. This is the right design for users, and importantly, for users of non-English languages. Hello world in C++ finally works.

— Strongly Favor

I support this proposal as I believe it provides important functionality despite continuing to have reservations regarding its use of the literal encoding as a proxy for the determination of the encoding of non-literal text. I also strongly dislike the `vprint_unicode()` and `vprint_nonunicode()` names; I think they would be better named `vprint_utf8()` and `vprint_binary()` or similar. I am very happy with the decision to treat encoding errors as UB (so that errors can be diagnosed) with normative guidance to substitute replacement characters in error scenarios.

— Weakly Favor

Baked enough; I continue to have reservations about how baked our format string syntax really is.

— Weakly Favor

This is a very useful piece of functionality; however, the interaction of the ostream overloads with the iostreams model (e.g., sentry object, exception mask) is unclear and will require additional elaboration.

— Weakly Favor

The number of defects that had to be fixed in `std::format` after C++20 shipped makes me cautious here.

— Weakly Favor

Probably it's good but if it's not the part of the standard I don't think it's a huge problem

— Neutral

This paper introduces functionality for interacting with a terminal ("If stream refers to a terminal capable of displaying Unicode, ...") that should be accessible on its own with a suitable minimal API and not require use of a large text formatting facility. The user might want to make processing depend on "if a terminal is capable of displaying Unicode" way before actually invoking "print".

— Strongly Against

This proposal is a logical extension of `std::format`, which I am generally in favor. But I have not participated enough in this proposals discussion to vote.

— Did Not Participate

§ 3.3. Poll 3: [P1467R7] Extended Floating-Point Types

This paper represents careful thought by both WG21 and WG14, as well as careful discussion by the C / C++ liaison committee. I participated in a C / C++ liaison committee discussion on this topic, and followed many of the other discussions. I think the authors carefully manage the divergence from C. Some divergence is inevitable. I do like that the authors have decided to keep the C arithmetic conversion rules, both to avoid divergence and to make the types of expressions more clear.

Regarding Section 5.5.2.1 ("Standard conversion of constants"), my preference would be for C++ to get rid of all such implicit conversions. I don't mind spelling out the type of a constant. Thus, I think the authors made the right choice.

Section 5.6 would prevent implementations from supporting mixed-precision arithmetic between two implementation-provided types whose conversion ranks are unordered. However, this would be a weird performance optimization at best. (For correctness, the best thing to do would be to cast everything to a type that can represent both types' sets of values.) Implementations could provide intrinsics to cover this case, if it's really needed.

Regarding Section 5.9, forbidding implicit pointer conversions between pointers to two different floating-point types with the same representation would help make code more portable, because the types might have different representations on some platforms. If code only needs to work on platforms where (for example) `double` and `std::float64_t` have the same representation, it can just `reinterpret_cast`.

Regarding Section 6.2, I am glad that we are requiring `from_chars` and `to_chars` to work for extended floating-point types.

Regarding Section 7.5, I agree with the authors' position that the new aliases shouldn't alias the standard types. I wish C++ had done the same thing with `int32_t`, etc. For example, it's painful to have to do explicit instantiation of a templated type for both `long` and `long long` (or deduce the type of `int64_t` at build time), when users really want to use the `int64_t` instantiation.

Regarding Section 9.2.2 ("Extended floating-point types"), I do agree that we should change or remove the Note. I would still vote SF for this proposal without the change, though.

I appreciate the drive-by wording corrections ("shall be" -> "is," etc.) in Section 9.2.5.

— Strongly Favor

Extended floating point units are proliferating in high performance computing and even more so in the machine learning/AI community.

— Strongly Favor

While, for practical reasons, implementations have avoided using extended integer types, having extended floating-point types is a clearly useful addition given modern hardware support. The library component of the proposal is relatively minor, but has been properly reviewed in light of the core-language changes (especially the question of literal suffixes) and C compatibility issues.

— Weakly Favor

This seems reasonable to me with the literals being a language feature. The lack of implementation experience is a bit concerning on the language side, however.

— Weakly Favor

This proposal has no implementation experience, yet it changes overload resolution rules. I deem that unacceptable; we can't pretend to be able to run overload resolution in our heads.

— Weakly Against

1. Lack of implementation experience, particularly on the to / from chars side
2. Design is still in flux. You came into LEWG with design questions, and LEWG pushed you in a direction that differed from the paper's current direction.
3. I believe reasonable alternatives were raised for how to do streaming formatting with the existing virtual functions, though the text SG would need to weigh in whether such an approach is reasonable.

— Weakly Against

I am glad extended floating point types got some attention and I welcome float16-128. I am slightly sceptical about standardizing bfloat16 though, since it is not an IEEE FP format. But I have not participated in any discussions.

— Did Not Participate

§ 3.4. Poll 4: [\[P2278R2\]](#) `cbegin` Should Always Return A Constant Iterator

This corrects a design mistake in `ranges::cmeow`. Not a fan of the last minute renaming but I can live with it.

— Strongly Favor

This should be a defect against C++20 and it is a must have to accept C++23 (we should fix major bugs before we add new stuff).

— Strongly Favor

Adding members to `span` looks like going in the completely wrong direction, but the `const_iterator/all_const` view are good additions consistent with the design of `ranges`.

— Weakly Favor

My expectation is that `cbegin` returns an iterator which can't be used for modification. It would be surprising to me to use `cbegin` and then pass that iterator onwards and allow modifying behavior.

— Weakly Favor

Several rather different approaches have been considered in this area. This one has the unique virtue of being a new, broadly applicable feature rather than an incomplete compatibility fix for `std::cbegin`. The changes to `std::ranges::cbegin` seem much more likely to fix bugs than to introduce them. The new `views::all_const` (whatever we name it) is very natural as a means of avoiding explicit iterators entirely.

— Weakly Favor

Weakly, because while the facility is important, the name `all_const` is terrible. It should be changed to `as_const`.

— Weakly Favor

The facility looks fine; the naming is bad (see [\[P2501R0\]](#)).

— Neutral

The name "all_const" sinks this proposal for me. See [\[P2501R0\]](#).

— Weakly Against

The name should be changed back. I would be in favour of it with the original name.

— Weakly Against

§ 3.5. Poll 5: [\[P2408R4\]](#) Ranges Iterators As Inputs To Non-Ranges Algorithms

This is a critical change for compatibility of ranges and non-ranges algorithms. In particular, parallel computing in Standard C++ currently depends on the non-ranges standard algorithms. Thus, this paper expands the scope of parallel computing in Standard C++, and lays a foundation for expanding the Standard's parallel computing features.

— Strongly Favor

This paper supports proxy iterators, but demotes all of them to `forward_iterator` only, so it may block the algorithm from benefiting from stronger access guarantees. The fully rangified algorithms, could provide more benefits. But, I believe blocking this paper, to wait for them, would be the case of "perfect being enemy of good".

— Weakly Favor

The imperfect correspondence between C++17 iterator categories and C++20 iterator concepts is a significant conceptual burden. Any improvement to their alignment is welcome even if it is incomplete so long as it is teachable, and "usable as read-only" is certainly that. Bearing in mind that generic functions outside the standard may be written in terms of the non-Ranges algorithms in the standard library, the additional utility (in new code) is even a bit larger than what is described.

— Weakly Favor

Vague concerns [from an implementer].

— Neutral

§ 3.6. Poll 6: [P2374R3] `views::cartesian_product`

This is an essential component for iterating with ranges and range-for loops over multidimensional index spaces.

— Strongly Favor

`views::cartesian_product` is particularly important for HPC as it can be used in combination with `views::iota` to express multi-dimensional index iteration.

— Strongly Favor

My favorite view. Too bad we didn't go the extra mile to support input iterator as the first parameter. The rest of the design is only logical.

— Strongly Favor

Nice simple addition to ranges. I had a number of use cases that could benefit from this functionality.

— Weakly Favor

I do not think this is the highest priority item on the list, but it is technically sound and useful.

— Weakly Favor

I think that `std::views::cartesian_product()` should produce (something equivalent to) `std::views::single(std::tuple<>())` rather than `std::views::empty<std::tuple<>>`. See [here](#); the analogy with `zip` is flawed because `zip` is not multiplicative and its special case would naturally be `std::ranges::views::repeat(std::tuple<>())` anyway. This could be resolved via an LWG issue or NB comment, but I would prefer that it be rectified in LEWG (and perhaps immediately sent for the last C++23 poll this month).

— Weakly Against

Ranges is such a mess already and anything but existing practice as we design on the fly instead of making the library mature and fix major bugs. This is not how standardization should happen in C++.

— Weakly Against

§ 3.7. Poll 7: [\[P2441R1\]](#) `views::join_with`

This is one of many papers that are helping to make C++ ranges more usable and complete.

— Strongly Favor

This is an important useful range adaptor that has field experience in 3rd party libraries.

— Strongly Favor

This is an important and useful range adaptor that has field experience in 3rd party libraries.

— Weakly Favor

A reasonable addition. I wish there was a way to use multiple elements as a separator but that probably belongs to a different API.

— Weakly Favor

Before we add more and more stuff to the ranges library, we should make it robust, look whether programmers are using it, and especially fix all major bugs.

— Weakly Against

It's probably correct as-is, but as it's a very minor feature, I haven't taken the time to participate in its discussions.

— Did Not Participate

§ 3.8. Poll 8: [\[P2446R1\]](#) `views::all_move`

This is a useful facility that is hard for users to provide themselves, and since it touches on move semantics it should be in the standard because it allows for more consistent code and makes teaching easier.

— Strongly Favor

This is one of many papers that are helping to make C++ ranges more usable and complete.

— Strongly Favor

Remaining conscious of the question of naming, this simple and composable component is ready for wording review and should be approved as a feature even if it's later altered.

— Weakly Favor

Weakly, because while the facility is important, the name `views::all_move` is terrible. It should be changed to `views::move`.

— Weakly Favor

The facility looks fine; the naming is bad (see [\[P2501R0\]](#)).

— Neutral

I can't vote in favour of this name. I would be in favour of it with the original name.

— Weakly Against

The functionality is useful, but the name is not valid English.

— Weakly Against

§ 3.9. Poll 9: [\[P2467R0\]](#) Support Exclusive Mode For `fstreams`

This helps make C++ more compatible with C, and exposes a useful feature in C++ (so that users do not need to resort to C in order to get it).

— Strongly Favor

I hand-roll this functionality in two different code bases now. Please standardize it.

— Strongly Favor

To the extent that we care about `fstream`, this change is long overdue.

— Weakly Favor

This is not the most common need, but it should be supported.

— Weakly Favor

§ 3.10. Poll 10: [\[P2438R1\]](#) `string::substr()` &&

This makes it easier to write efficient string manipulation code, and makes such code look more "functional." The authors carefully limit scope to `substr`, and explain why they didn't change other functions. They also talk about how to maintain ABI stability.

— Strongly Favor

Certainly the opportunity to improve the performance of existing code is attractive, and the cases that break are surely rare (except perhaps in certain Hyrum's-law usage of generic code that uses `forward`). However, a disadvantage of such compatibility is that it is possible to have the silent change be to increase memory usage if a small substring is taken of a large rvalue string.

Implementations might choose to copy for small size ratios (especially if the starting position is not 0) to avoid this, but they might also hope that clients would use `shrink_to_fit` in such a case, which old clients would not know to do.

— Weakly Favor

Through many years of profiling and code reviews I frequently see inefficient code due to the use of `substr`. I lean towards banning it in my code, especially when `string_view` often the better/right tool. However, there are cases where `string_view` cannot be used because of dangling storage and this proposal provides a way to resolve at least some of these cases.

— Weakly Favor

Good but not sufficiently important.

— Neutral

§ 3.11. Poll 11: [\[P2396R0\]](#) Concurrency TS v2 Fixes

These are minor fixes that should be uncontroversial.

— Strongly Favor

These fixes are trivial and obviously correct in direction. Approving the direction will improve implementation consistency even if the paper has not yet been merged into the TS (or the other papers in flight).

— Weakly Favor

I don't like the fine-grained headers for every feature, but the rest of the changes are essential.

— Weakly Favor

§ 3.12. Poll 12: [\[P2198R3\]](#) Freestanding Feature-Test Macros And Implementation-Defined Extensions C++23

Feature test macros are useful for keeping code portable over different compiler versions, but they only work if they do not lie.

— Strongly Favor

The policy looks good. I like the solution with single macro, that informs is the feature test macro feature is available, given reports that some libraries already properly disable them.

— Strongly Favor

Freestanding users have a need to know what facilities are available in code that might be used in both hosted and freestanding modes, and this gives implementations a way to express that information in a conforming manner.

— Weakly Favor

This seems reasonable, but implementers' voices are more relevant than mine here.

— Weakly Favor

If this helps people use freestanding, it's good, but I don't know enough myself.

— Neutral

I don't quite understand this poll; R4 of the paper is available, and changes the direction significantly.

— Weakly Against

I object to bumping the values of existing macros. The rationale given for that is flawed. Making the macros optional for freestanding should simply be fixed as a DR for the current standard. The other changes are good, but coupled to unnecessary churn with incorrect rationale.

— Weakly Against

I would prefer to have more granular feature test macros than for code to have to be explicitly written specifically for hosted vs freestanding.

— Strongly Against

Discussion on the reflector has meanwhile revealed that the bulk-bump of the feature test macros is not needed; implementations already do the right thing (and future releases will fix the remaining bugs).

— Strongly Against

§ References

§ Informative References

[P0009R14]

Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. *MDSPAN*. 15 November 2021. URL: <https://wg21.link/p0009r14>

[P0592R4]

Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: <https://wg21.link/p0592r4>

[P1467R7]

David Olsen, Michał Dominiak, Ilya Burylov. *Extended floating-point types and standard names*. 24 November 2021. URL: <https://wg21.link/p1467r7>

[P2093R11]

Victor Zverovich. *Formatted output*. 6 December 2021. URL: <https://wg21.link/p2093r11>

[P2198R3]

Ben Craig. *Freestanding Feature-Test Macros and Implementation-Defined Extensions*. 12 November 2021. URL: <https://wg21.link/p2198r3>

[P2278R2]

Barry Revzin. *cbegin should always return a constant iterator*. 17 November 2021. URL: <https://wg21.link/p2278r2>

[P2374R3]

Sy Brand, Michał Dominiak. *views::cartesian_product*. 13 December 2021. URL: <https://wg21.link/p2374r3>

[P2396R0]

David Goldblatt. *Concurrency TS 2 fixes*. 14 June 2021. URL: <https://wg21.link/p2396r0>

[P2408R4]

David Olsen. *Ranges iterators as inputs to non-Ranges algorithms*. 16 November 2021. URL: <https://wg21.link/p2408r4>

[P2438R1]

Federico Kircheis, Tomasz Kamiński. [*std::string::substr\(\) &&*](#). 20211130. URL: <https://wg21.link/p2438r1>

[P2441R1]

Barry Revzin. [*views::join_with*](#). 17 November 2021. URL: <https://wg21.link/p2441r1>

[P2446R1]

Barry Revzin. [*views::all_move*](#). 17 November 2021. URL: <https://wg21.link/p2446r1>

[P2456R0]

Bryce Adelstein LeBach. [*2021 December Library Evolution Polls*](#). 8 December 2021. URL: <https://wg21.link/p2456r0>

[P2467R0]

Jonathan Wakely. [*Support exclusive mode for fstreams*](#). 15 November 2021. URL: <https://wg21.link/p2467r0>

[P2501R0]

Ville Voutilainen. [*Undo the rename of views::move and views::as_const*](#). 14 December 2021. URL: <https://wg21.link/p2501r0>